

```

# =====
# Student Name : Srushti Kale
# PRN          : 23070521067
# Experiment 2 : Housing Price Prediction using Deep Feed Forward Network
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error

# --- 1. Load Dataset ---
# Srushti Kale - 23070521067: Loading standard California Housing dataset
housing = fetch_california_housing(as_frame=True)
X = housing.data
y = housing.target

print("Dataset Shape:", X.shape)
print("Features:", housing.feature_names)

# --- 2. Data Preprocessing & Splitting ---
# Srushti Kale - 23070521067: Splitting data into train and test sets (80-20 % split)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Srushti Kale - 23070521067: Normalizing features using StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# --- 3. Design Deep Feed Forward Network (DFFN) Architecture ---
# Srushti Kale - 23070521067: Building a multi-layer deep feed forward regression model
model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    keras.layers.Dense(64, activation='relu', name="Dense_2"),
    keras.layers.Dense(32, activation='relu', name="Dense_3"),
    keras.layers.Dense(1, name="Output_Layer") # Single continuous output for regression
])

model.summary()

# --- 4. Model Compilation ---
# Srushti Kale - 23070521067: Compiling with Adam optimizer and MSE loss
model.compile(
    optimizer=keras.optimizers.Adam(learning_rate=0.001),
    loss='mean_squared_error',
    metrics=['mae']
)

```

```

)

# --- 5. Model Training ---
# Srushti Kale - 23070521067: Fitting the model with a validation split of 20%
history = model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=40,
    batch_size=32,
    verbose=1
)

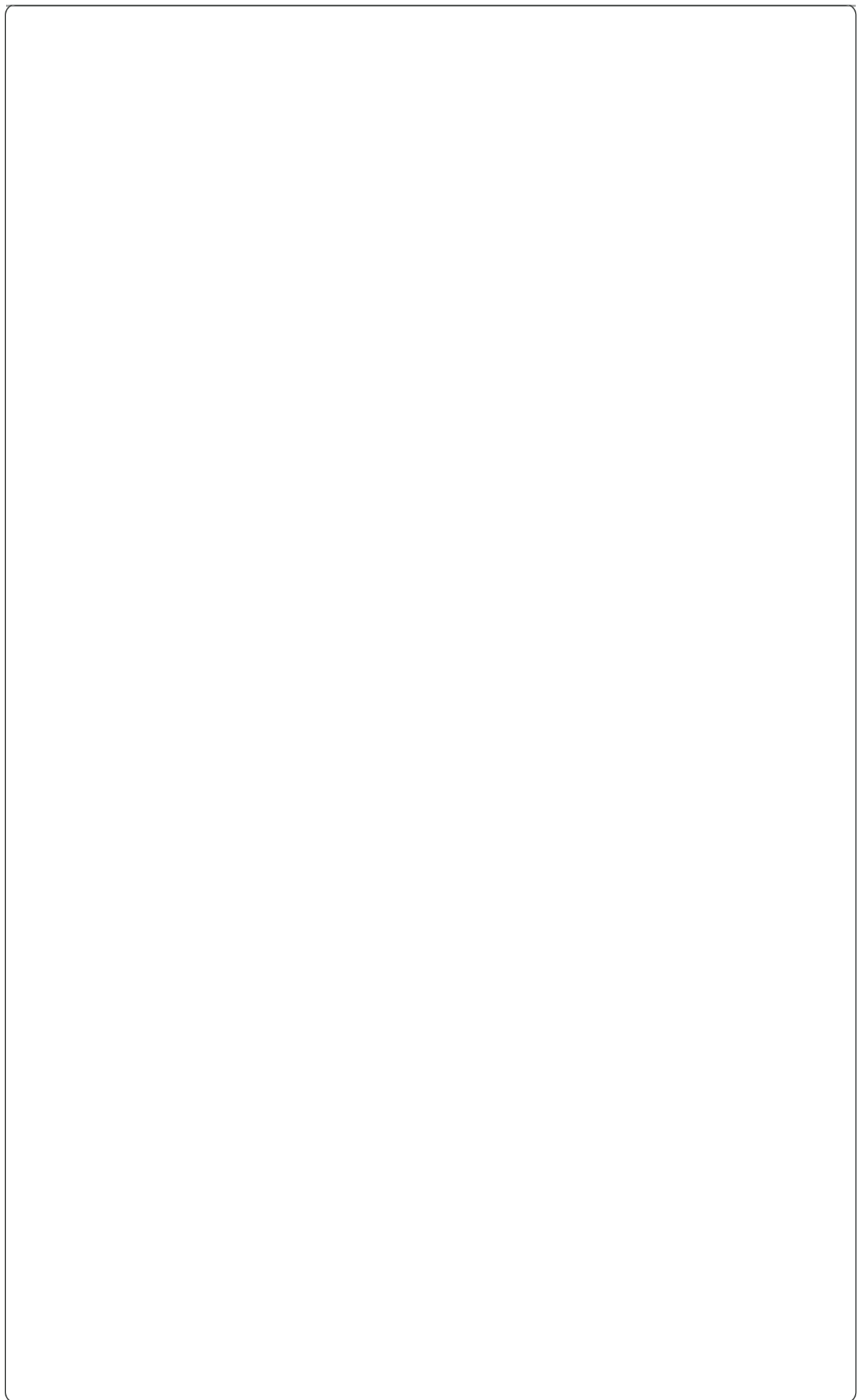
# --- 6. Model Evaluation (MAE, MSE, RMSE) ---
# Srushti Kale - 23070521067: Generating predictions on the test set
y_pred = model.predict(X_test_scaled).flatten()

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print("\n--- Evaluation Metrics (Srushti Kale - 23070521067) ---")
print(f"Mean Absolute Error (MAE) : {mae:.4f}")
print(f"Mean Squared Error (MSE) : {mse:.4f}")
print(f"Root Mean Squared Error (RMSE): {rmse:.4f}")

# --- 7. Visualization: Predicted vs Actual ---
# Srushti Kale - 23070521067: Plotting actual vs predicted values
plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_pred, alpha=0.3, color='blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], '--r', linestyle='solid')
plt.title("Actual vs Predicted House Prices - Srushti Kale (23070521067)")
plt.xlabel("Actual Values")
plt.ylabel("Predicted Values")
plt.grid(True)
plt.show()

```



Dataset Shape: (20640, 8)

Features: ['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'Ave
/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106:
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential"

Layer (type)	Output Shape	Param #
Dense_1 (Dense)	(None, 128)	1,152
Dense_2 (Dense)	(None, 64)	8,256
Dense_3 (Dense)	(None, 32)	2,080
Output_Layer (Dense)	(None, 1)	33

Total params: 11,521 (45.00 KB)

Trainable params: 11,521 (45.00 KB)

Non-trainable params: 0 (0.00 B)

Epoch 1/40

413/413 ————— 12s 14ms/step - loss: 0.6080 - mae: 0.5436 - val_

Epoch 2/40

413/413 ————— 5s 11ms/step - loss: 0.3660 - mae: 0.4315 - val_

Epoch 3/40

413/413 ————— 5s 12ms/step - loss: 0.3509 - mae: 0.4151 - val_

Epoch 4/40

413/413 ————— 2s 5ms/step - loss: 0.3265 - mae: 0.3999 - val_

Epoch 5/40

413/413 ————— 1s 2ms/step - loss: 0.3121 - mae: 0.3888 - val_

Epoch 6/40

413/413 ————— 1s 2ms/step - loss: 0.3100 - mae: 0.3821 - val_

Epoch 7/40

413/413 ————— 1s 2ms/step - loss: 0.2976 - mae: 0.3755 - val_

Epoch 8/40

413/413 ————— 1s 2ms/step - loss: 0.2947 - mae: 0.3730 - val_

Epoch 9/40

413/413 ————— 1s 2ms/step - loss: 0.2882 - mae: 0.3703 - val_

Epoch 10/40

413/413 ————— 1s 2ms/step - loss: 0.2840 - mae: 0.3677 - val_

Epoch 11/40

413/413 ————— 2s 4ms/step - loss: 0.2790 - mae: 0.3620 - val_

Epoch 12/40

413/413 ————— 2s 5ms/step - loss: 0.2760 - mae: 0.3615 - val_

Epoch 13/40

413/413 ————— 1s 3ms/step - loss: 0.2765 - mae: 0.3600 - val_

Epoch 14/40

413/413 ————— 1s 2ms/step - loss: 0.2749 - mae: 0.3592 - val_

Epoch 15/40

413/413 ————— 1s 2ms/step - loss: 0.2696 - mae: 0.3567 - val_

Epoch 16/40

413/413 ————— 1s 2ms/step - loss: 0.2658 - mae: 0.3520 - val_

Epoch 17/40

413/413 ————— 1s 2ms/step - loss: 0.2648 - mae: 0.3505 - val_

Epoch 18/40

413/413 ————— 1s 2ms/step - loss: 0.2589 - mae: 0.3487 - val_

Epoch 19/40

413/413 ————— 1s 2ms/step - loss: 0.2579 - mae: 0.3470 - val_

Epoch 20/40

413/413 ————— 1s 2ms/step - loss: 0.2559 - mae: 0.3448 - val_

Epoch 21/40

Epoch 21/40

413/413 ————— 1s 2ms/step - loss: 0.2550 - mae: 0.3460 - val_

Epoch 22/40

413/413 ————— 1s 2ms/step - loss: 0.2495 - mae: 0.3417 - val_

Epoch 23/40

413/413 ————— 1s 3ms/step - loss: 0.2501 - mae: 0.3397 - val_

Epoch 24/40

413/413 ————— 1s 4ms/step - loss: 0.2487 - mae: 0.3407 - val_

Epoch 25/40

413/413 ————— 1s 3ms/step - loss: 0.2482 - mae: 0.3399 - val_

Epoch 26/40

413/413 ————— 1s 2ms/step - loss: 0.2425 - mae: 0.3346 - val_

Epoch 27/40

413/413 ————— 1s 2ms/step - loss: 0.2419 - mae: 0.3350 - val_

Epoch 28/40

413/413 ————— 1s 2ms/step - loss: 0.2405 - mae: 0.3343 - val_

Epoch 29/40

413/413 ————— 1s 2ms/step - loss: 0.2395 - mae: 0.3322 - val_

Epoch 30/40